SESSION 3: ADVANCED RESOURCES AND PROVIDERS

FINDMORE ACADEMY

Interno

Multiple Providers

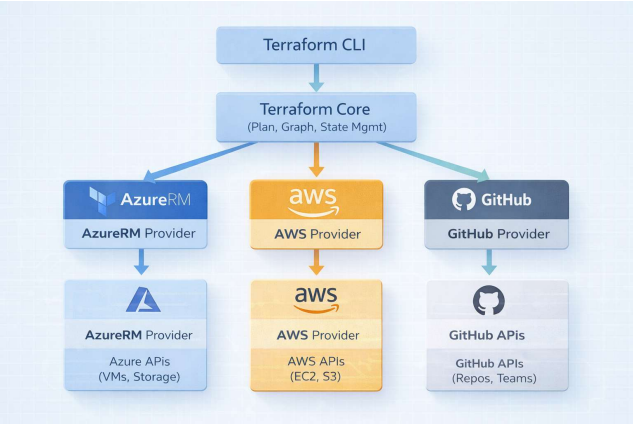
- Terraform can use multiple providers within the same configuration.
- Providers represent different platforms or APIs (clouds, SaaS, on-prem).
- Enables:
  - Multi-cloud deployments
  - Hybrid architectures
  - Infrastructure + SaaS management in a single workflow
- The same provider can be configured multiple times using aliases.
- Examples:
  - Azure + AWS in one configuration
  - AzureRM + AzureCAF
  - Azure + GitHub + Kubernetes

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
    github = {
      source  = "integrations/github"
      version = "~> 6.0"
    }
  }
}

# Azure Provider
provider "azurerm" {
  features {}
}

# GitHub Provider
provider "github" {
  token = var.github_token
  owner = var.github_owner
}

# Azure Resource
resource "azurerm_resource_group" "rg" {
  name     = "rg-terraform-demo"
  location = "East US"
}

# GitHub Resource
resource "github_repository" "repo" {
  name        = "terraform-demo-repo"
  description = "Repository managed by Terraform"
  visibility  = "private"
}
```

Interno

How Terraform Interacts with APIs

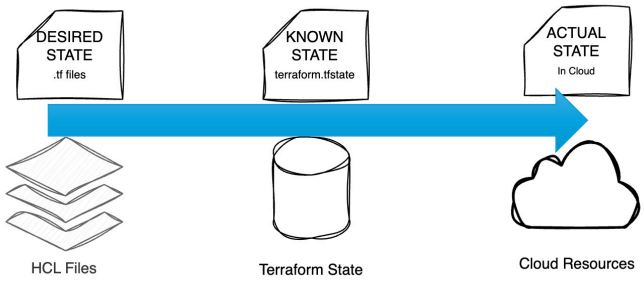
- Terraform does not manage infrastructure directly.
- It interacts with platforms exclusively through provider APIs.
- Each provider:
  - Translates Terraform resources into API calls
  - Handles authentication and authorization
- Terraform Core:
  - Builds the execution plan
  - Determines dependencies
  - Orchestrates API calls in the correct order



FINDMORE ACADEMY

Interno

Terraform State File



The Terraform state file (**terraform.tfstate**) tracks the current known state of managed infrastructure.

- Acts as Terraform's **source of truth**, mapping real-world resources to Terraform configurations.
- Enables Terraform to determine:
  - What resources exist
  - What changes are required
  - What can be safely updated or destroyed
- Used during:
  - terraform plan to calculate differences (diff)
  - terraform apply to execute required changes

Why State is Important

- Allows **idempotent operations** by comparing desired vs actual state.
- Detects **configuration drift** caused by manual or external changes.
- Supports dependency resolution and accurate resource ordering.

State & Drift

- Drift occurs when infrastructure changes outside Terraform.
- Terraform detects drift during plan and proposes corrective actions.

FINDMORE ACADEMY

Interno

Terraform Refresh

- Reads the current settings from all managed remote objects and updates the Terraform state to match.
- This does not modify your real remote objects, but it modifies the Terraform state.
- OBS: This command is deprecated. Instead, add the -refresh-only flag to terraform apply and terraform plan commands.

> terraform apply -refresh-only

```
azureuser@vm-windows:~/Terraform$ terraform apply -refresh-only
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/[redacted]/resourceGroups/demo-rg]
azurerm_storage_account.storage: Refreshing state... [id=/subscriptions/[redacted]/resourceGroups/demo-rg/providers/Microsoft.Storage/storageAccounts/fmacademystorage12345]
azurerm_storage_blob.index: Refreshing state... [id=https://fmacademystorage12345.blob.core.windows.net/$web/index.html]

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have affected this plan:

  # azurerm_resource_group.rg has changed
  ~ resource "azurerm_resource_group" "rg" {
      id       = "/subscriptions/[redacted]/resourceGroups/demo-rg"
      name     = "demo-rg"
    + tags     = {}
      # (2 unchanged attributes hidden)
    }

  # azurerm_storage_account.storage has changed
  ~ resource "azurerm_storage_account" "storage" {
      id                               = "/subscriptions/[redacted]/resourceGroups/demo-rg/providers/Microsoft.Storage/storageAccounts/fmacademystorage12345"
      name                             = "fmacademystorage12345"
    + tags                             = {}
      # (97 unchanged attributes hidden)

      # (4 unchanged blocks hidden)
    }
```


Terraform state commands

> Terraform state list

List resources contained in the terraform state file

> Terraform state show [resource_type.resource_name]

Displays the complete configuration stored of the specified resource(s) on the tf state file.

> Terraform state rm [resource_type.resource_name]

Remove a resource from state file. Terraform wil not manage the resource anymore.

```
azureuser@vm-windows:~/Terraform$ terraform state list
azurerm_resource_group.rg
azurerm_storage_account.storage
azurerm_storage_blob.index
azureuser@vm-windows:~/Terraform$ terraform state show azurerm_storage_account.storage
# azurerm_storage_account.storage:
resource "azurerm_storage_account" "storage" {
    access_tier                     = "Hot"
    account_kind                    = "StorageV2"
    account_replication_type        = "LRS"
    account_tier                    = "Standard"
    allow_nested_items_to_be_public = true
    allowed_copy_scope              = null
    cross_tenant_replication_enabled = true
    default_to_oauth_authentication = false
    dns_endpoint_type               = "Standard"
    edge_zone                       = null
    enable_https_traffic_only       = true
    https_traffic_only_enabled      = true
    id                              = "/subscriptions/[redacted]/resourceG
    infrastructure_encryption_enabled = false
    is_hns_enabled                  = false
    large_file_share_enabled        = false
    local_user_enabled              = true
    location                        = "westeurope"
    min_tls_version                 = "TLS1_2"
    name                            = "fmacademystorage12345"
    nfsv3_enabled                   = false
    primary_access_key              = (sensitive value)
    primary_blob_connection_string  = (sensitive value)
    primary_blob_endpoint           = "https://fmacademystorage12345.blob.core.windows.net/"
```

Terraform State: Backend Types

What Is a Backend?

- Defines where and how Terraform state is stored
- Controls state storage, locking, and collaboration
- Configured in the terraform { backend {} } block

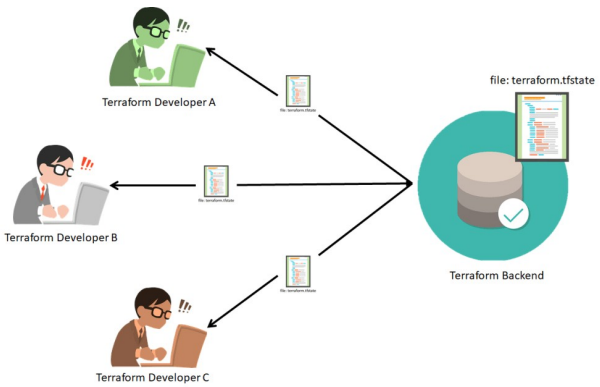
Main Backend Categories

1. Local Backend

- Stores state on the **local filesystem**
- Default backend if none is configured
- No locking or remote collaboration (each user has own state file

2. Remote Backends

- State stored in a **remote system**
- Enables **team collaboration**
- Supports **state locking** and **consistency**



Fonte: https://digitalvarys.com/complete-terraform-tutorial-part-6-terraform-backends/

FINDMORE ACADEMY

Interno

Remote Backends

Common Remote Backend Types

- Azure Blob Storage (azurerm)
- AWS S3 (with DynamoDB for locking)
- Google Cloud Storage (GCS)
- Terraform Cloud / Terraform Enterprise
- HTTP / Consul / PostgreSQL (less common)

Why Use Remote Backends?

- Prevents state conflicts
- Enables safe parallel execution
- Centralized source of truth
- Required for CI/CD pipelines

Changing backend requires:
> terraform init -migrate-state

backend.tf

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "tfstateprod001"
    container_name       = "tfstate"
    key                  = "terraform.tfstate"
  }
}
```

```
Successfully configured the backend "azurerm"! Terraform will automatically
use this backend unless the backend configuration changes.
Initializing provider plugins...
- Reusing previous version of hashicorp/random from the dependency lock file
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/random v3.7.2
- Using previously-installed hashicorp/azurerm v3.117.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

FINDMORE ACADEMY

Interno

State Locking

What It Is

A mechanism that prevents multiple Terraform runs from modifying the same state at the same time

Why It Matters

Avoids:

- State corruption
- Race conditions
- Lost or overwritten infrastructure changes

How It Works

Implemented by the **remote backend**

Automatically acquired during:

- terraform plan
- terraform apply
- terraform destroy

```
● → ec2 git:(main) ✗ terraform plan
  Acquiring state lock. This may take a few moments...

  Terraform used the selected providers to generate the following
  execution plan. Resource actions are indicated with the following
  symbols:
    + create

  Terraform will perform the following actions:

    # aws_instance.terraform-state-test will be created
    + resource "aws_instance" "terraform-state-test" {
        + ami                                  = "ami-0cf2b4e024cdb6960"
        + arn                                  = (known after apply)
        + associate_public_ip_address          = (known after apply)
        + availability_zone                    = (known after apply)
        + cpu_core_count                       = (known after apply)
        + cpu_threads_per_core                 = (known after apply)
        + disable_api_stop                     = (known after apply)
        + disable_api_termination              = (known after apply)
        + ebs_optimized                        = (known after apply)
        + get_password_data                    = false
        + host_id                              = (known after apply)
```

Fonte: https://devopscube.com/setup-terraform-remote-state-s3-dynamodb/

Data Block

data { }

- data blocks allow Terraform to read information from existing infrastructure.
- Useful for retrieving resources not managed directly by Terraform.
- Can query cloud services, APIs, or existing resources.
- Often used to reference shared resources in multi-module or multi-team environments.

```
# Data-sourcing Resources
data "azurerm_resource_group" "rg-vnet" {
  name = "RG-VNET"
}

data "azurerm_virtual_network" "vnet_example" {
  name = "vnet-shared-resources"
  resource_group_name = data.azurerm_resource_group.rg-vnet.name
}

# Creating a new resource (subnet) referencing the retrieved data
resource "azurerm_subnet" "subnet-example" {
  name = "sub-01"
  virtual_network_name = data.azurerm_virtual_network.vnet_example.name
  resource_group_name = data.azurerm_resource_group.rg-vnet
  address_prefixes = ["10.10.0.1/24"]

}
```

Terraform Import Command

What Is Terraform Import?

- Used to bring existing infrastructure under Terraform management
- Maps an already-created resource to a Terraform resource block
- Updates the state file only
- Does not generate Terraform code

Key Characteristics

- Resource must already exist in the target platform
- Requires a matching resource block in the .tf files
- Helps eliminate manual drift
- Commonly used during IaC adoption or migration projects

Important Limitations

- Terraform does not infer best-practice configuration
- Imported resources often require manual code adjustment
- Not all resource attributes are always imported

```
resource "azurerm_linux_virtual_machine" "vm1" {
    name = "vm1"
    resource_group_name = "vm-test-westeurope"
    location = "westeurope"
    size                = "Standard_B2ls_v2"
    admin_username      = "azureuser"
    admin_password      = "Password1234!"
    zone = "1"
    os_disk {
        caching              = "ReadWrite"
        storage_account_type = "Standard_LRS"
  }

    network_interface_ids = ["vm1471_z1"]
    source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "24_04-lts"
    version   = "latest"
  }
}
```

```
~/Terraform/vm_import$ terraform import azurerm_linux_virtual_machine.vm1 /subscriptions/aa21ef4d-47c3-4cbf-a9b6-09980dd62132/resc
pe/providers/Microsoft.Compute/virtualMachines/vm1
azurerm_linux_virtual_machine.vm1: Importing from ID "/subscriptions/aa21ef4d-47c3-4cbf-a9b6-09980dd62132/resourceGroups/vm-test-w
ft.Compute/virtualMachines/vm1"...
azurerm_linux_virtual_machine.vm1: Import prepared!
  Prepared azurerm_linux_virtual_machine for import
azurerm_linux_virtual_machine.vm1: Refreshing state... [id=/subscriptions/aa21ef4d-47c3-4cbf-a9b6-09980dd62132/resourceGroups/vm-t
crosoft.Compute/virtualMachines/vm1]

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.
```

FINDMORE ACADEMY

Interno

Resource Dependency

What Is Resource Dependency in Terraform?

- Defines the order in which resources are created, updated, or destroyed
- Ensures infrastructure is provisioned safely and correctly
- Terraform builds a dependency graph automatically at runtime

Types of Dependencies:

- Implicit Dependencies

- Created when a resource references another resource's attributes

- Explicit Dependencies

- Defined using depends_on
- Used when no direct attribute reference exists
- Makes dependencies intentional and visible

Why Dependencies Matter

- Prevents race conditions
- Ensures correct provisioning order
- Enables parallel execution where possible

```
resource "azurerm_network_interface" "example" {
  name                = "example-nic"
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.example.id
    private_ip_address_allocation = "Dynamic"
  }
}

resource "azurerm_windows_virtual_machine" "vm-example" {
  name                = "example-machine"
  resource_group_name = azurerm_resource_group.example.name       # implicit dependencies
  location            = azurerm_resource_group.example.location
  size                = "Standard_F2"
  admin_username      = "adminuser"
  admin_password      = "P@$$w0rd1234!"
  network_interface_ids = [
    azurerm_network_interface.example.id,
  ]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2016-Datacenter"
    version   = "latest"
  }

  # explicit dependency
  depends_on = [ azurerm_network_interface.example ]
}
```

Interno

Dynamic Blocks

What Are Dynamic Blocks?

- Used to generate nested blocks programmatically.
- Help reduce code duplication in complex resources.
- Commonly used when the number of nested blocks is variable or data-driven.

Key Characteristics

- Defined using the dynamic keyword.
- Usually combined with:
  - for_each
  - Lists, maps, or objects
- Applies only to nested blocks, not top-level resources.

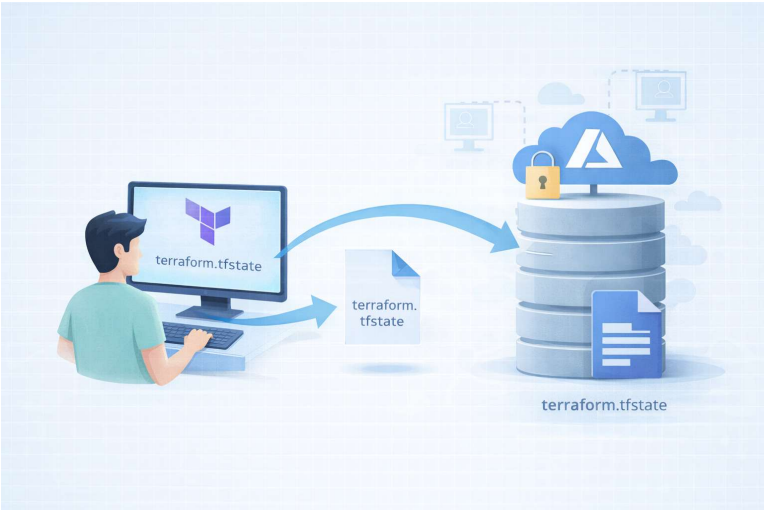
When to Use Dynamic Blocks

- Optional or repeated nested configurations
- Environment-specific settings
- Data-driven infrastructure definitions

```
# input variable with values
variable "nsg_rules" {
  description = "List of inbound NSG rules"
  type = list(object({
    name     = string
    priority = number
    port     = number
  }))

  default = [
    {
      name     = "allow-ssh"
      priority = 100
      port     = 22
    },
    {
      name     = "allow-http"
      priority = 200
      port     = 80
    }
  ]
}
# resource to be created
resource "azurerm_network_security_group" "nsg" {
  name                = "nsg-dynamic-demo"
  location            = "East US"
  resource_group_name = "rg-demo"

# "security_rule" resource block will be filled automatically with the variables content
  dynamic "security_rule" {
    for_each = var.nsg_rules
    content {
      name                       = security_rule.value.name
      priority                   = security_rule.value.priority
      direction                  = "Inbound"
      access                     = "Allow"
      protocol                   = "Tcp"
      source_port_range          = "*"
      destination_port_range     = security_rule.value.port
      source_address_prefix      = "*"
      destination_address_prefix = "*"
    }
  }
}
```

Interno

Lab 3: Configuring Remote Backend in Azure and use of Terraform Import command



50

Interno